

Spark Repair Estimator Writeup

The biggest design decision I made was to keep the estimator flow card-based and swipe-driven instead of turning it into a long form. That matters because this app is meant for field use. An acquisition agent is standing in an empty house, usually on a phone, trying to make fast decisions without losing track of what has already been reviewed. The swipe layout keeps the current group visible, makes the next action obvious, and gives the app a more natural rhythm for walkthroughs than a standard checklist would.

The strongest part of the build is the way the app covers the core contest workflow without feeling like a demo shell. It supports multiple projects, local persistence, room instances, repair groups, photo capture, progress tracking, and export. I also tried to keep the UI clear enough that someone could open it cold and understand the flow quickly. The totals are always visible, the sections are organized around how a property is actually walked, and the export is built to be useful outside the app.

The most fragile part of the build is still the browser environment itself. This is a single-file front-end app, so it depends on local browser storage and CDN-loaded libraries. That makes it easy to review and easy to launch, but it also means large photo sets can hit storage limits and offline behavior depends on the cached app shell. OCR is also best-effort rather than guaranteed, so serial-number detection should be treated as a helpful extra, not a core dependency.

My creative addition was the serial-number helper for equipment photos. Since agents already take photos of HVAC units, water heaters, and appliances, trying to extract a serial number from the image gives the walkthrough more value than just storing the picture. I also added supporting tools like compare and contractor views so the app feels closer to something a real team could grow into, not just a one-off estimator.

If I had two more days, I would move photos out of `localStorage`, harden offline caching so it does not depend on remote assets, and tighten a few edge cases around custom items and room management. I would also spend more time on export formatting and small usability details, because those are the kinds of things that make a submission feel finished.

I used AI as a development aid for cleanup, layout refinement, and review, but I kept the final direction focused on the contest brief and the actual field workflow.